

"架构辩论赛"记录

团队名称：哪一队

日期：2026-04-28

文档版本：V1.0

一、实验背景

根据 Phase 2 任务要求，团队4名成员分别采用不同的 Prompt 策略向 AI 咨询架构建议，旨在对比不同提问方式对 AI 输出质量的影响，并基于 AI 建议进行团队决策。

项目约束（所有成员共享）：

- 团队规模：4人大学生团队
- 开发周期：10周
- 技术储备：熟悉 Java 语言
- 目标用户：单校大学生，初期并发 ≤ 3000
- 核心功能：快递代取、二手交易、组队匹配
- 预算：学生项目，无商业预算

二、实验记录

2.1 成员A：韩序 — 直接提问策略

Prompt原文：> 请推荐一个校园互助服务平台的架构

AI回答摘要：

AI 直接推荐了**前后端分离 + 微服务架构**，技术栈为 Go + Gin 框架，核心存储 MySQL + Redis，服务拆分为用户、任务、订单、支付、消息、评价等独立微服务，使用 Nacos 做服务发现，PostgreSQL + PostGIS 处理地理位置，WebSocket 做消息推送。

AI 同时提到："日均千单以下可直接单体部署（Laravel/Django）；超过千单按上述微服务拆分"。

AI输出特点：

- 默认假设为有一定经验的开发团队
- 直接推荐当前业界"流行"的微服务架构
- 未询问团队规模、技术储备、时间约束等关键信息
- 技术栈推荐 Go 语言，与团队 Java 背景不符
- 包含大量高级组件（Nacos、PostGIS、RabbitMQ 延时消息），对 4 人学生团队明显过度设计

2.2 成员B：陈朵朵 — 提供详细约束后提问

Prompt原文： > 我们是一个4人大学生团队，10周时间，技术栈Java+Vue，需要开发一个支持快递代取、二手交易、组队匹配的校园互助平台，单校用户3000人以内。请推荐合适的架构并说明理由

AI回答摘要：

AI 明确推荐**前后端分离 + 分层单体架构**（Spring Boot + Vue + MySQL + Redis），并给出了详细的理由：

要点	内容
架构风格	分层单体架构，非微服务
技术栈	Java 17 + Spring Boot 2.7+、Vue 3 + Vite + Pinia、MySQL 8.0、Redis
模块划分	按业务包结构划分：DeliveryController、SecondhandController、TeamController、UserController
关键决策	放弃即时 IM 和地图轨迹跟踪，用订单留言代替；不做实时聊天系统
部署方案	阿里云 ECS 2核4G + Nginx，Docker 运行 MySQL/Redis
里程碑	1-2周脚手架 → 3-5周核心功能 → 6-7周组队模块 → 8周联调 → 9周部署测试 → 10周文档答辩

AI 明确对比了微服务的缺点：“需要注册中心、配置中心、APM监控，增加约3-4倍开发量；分布式事务、服务间调用会极大增加bug风险”。

AI输出特点：

- 严格基于提供的约束条件给出建议
- 主动排除了微服务，推荐与团队技术栈匹配的方案
- 提供了可落地的详细模块划分和开发里程碑
- 包含务实的取舍建议（放弃 IM、简化匹配算法）
- 输出质量高，可直接作为项目参考

2.3 成员C：王雨婷 — 要求对比多种方案

Prompt原文： > 请对比分层架构、MVC、微服务、事件驱动架构在校园互助平台场景下的优劣，从开发复杂度、可维护性、可扩展性、性能、学习成本、10周可行性6个维度分析

AI回答摘要：

AI 对四种架构进行了系统对比：

架构	核心结论
分层架构	开发简单、学习成本极低、10周落地无压力；但代码耦合度高，长期扩展能力差
MVC架构	职责划分清晰、开发难度低、10周落地可行性最高；属于单体架构，大规模高并发拓展上限不高

架构	核心结论
微服务架构	解耦性强、扩展性好；但开发复杂度极高、学习成本远超本科基础、10周仅搭建基建就耗时过半， 坚决不选用
事件驱动架构	异步解耦、抗并发能力强；但开发难度高、需消息队列中间件、异步链路不直观、 不适合选用

最终结论：MVC架构为最优选择，兼顾开发效率、维护难度和落地保障；基础薄弱可备选分层架构；微服务和事件驱动技术过重、工期不匹配。

AI输出特点：

- 结构化对比清晰，维度明确
- 主动排除了两种架构（微服务、事件驱动），缩小了选择范围
- 但存在概念混淆：将"分层架构"和"MVC"作为并列选项对比，实际上 MVC 是一种表现层模式，常与分层架构结合使用
- 未给出具体技术栈建议，偏理论分析

2.4 成员D：罗昕雅 — 让AI扮演架构师角色

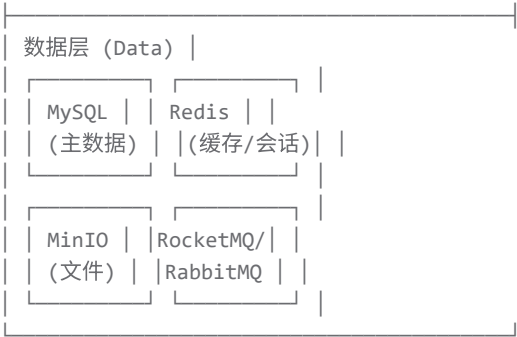
Prompt原文： > 你是一位有10年经验的软件架构师，曾为多个高校信息化项目设计架构。现在一个4人大学生团队要开发校园互助平台，请从架构师角度给出设计方案，包括架构风格、模块划分、技术选型建议

AI回答摘要：

AI 以"资深架构师"身份给出了完整方案：

架构风格： 分层单体 + 前后端分离





技术选型：

- 后端：Spring Boot 3.x + Spring Security + MyBatis-Plus + MySQL 8.0 + Redis + RabbitMQ
- 前端：React 18 + Ant Design Pro（管理后台）、Taro 3（小程序）
- 基础设施：阿里云 ECS 2核4G + RDS + OSS

模块划分：用户中心、任务中心、即时通讯、支付结算、评价举报、运营管理共6个模块

关键决策：

问题	决策	理由
微服务还是单体？	单体	4人团队，业务简单，分布式复杂度不划算
要不要分库分表？	不需要	高校场景日活<1万，单库轻松支撑
支付怎么实现？	模拟钱包+微信转账	真实支付需企业资质，初期可人工审核提现
实时通讯？	WebSocket长连接	聊天场景，Spring Boot + STOMP协议

AI输出特点：

- 输出最为完整，包含架构图、模块划分、技术选型、数据模型、团队分工、演进路线
- 以"架构师"身份主动做了大量工程判断（如放弃真实支付、选择单体架构）
- 但存在过度设计倾向：推荐了 RabbitMQ、XXL-JOB、MinIO、Elasticsearch 等组件，对 10 周项目略显沉重
- 前端技术栈推荐 React，与团队 Vue 背景不完全匹配
- 包含大量企业级实践（SonarQube、内容安全服务），学生项目难以落地

三、不同Prompt策略下AI回答质量对比

对比维度	成员A：直接问	成员B：给约束	成员C：对比方案	成员D：扮演架构师
架构建议合理性	★★ 差（推荐微服务+Go，与团队不匹配）	★★★★★ 优秀（精准匹配约束）	★★★★ 良好（排除明显不合适的）	★★★ 一般（合理但过度设计）

对比维度	成员A：直接问	成员B：给约束	成员C：对比方案	成员D：扮演架构师
技术栈匹配度	★★★ 差（Go 语言）	★★★★★ 优秀（Java+Vue）	★★★ 一般（未指定技术栈）	★★★ 一般（React 而非 Vue）
落地可行性	★★★ 差（Nacos、PostGIS 等过重）	★★★★★ 优秀（提供10周里程碑）	★★★★★ 良好（明确排除不可行的）	★★★ 一般（组件过多，10周难完成）
输出完整性	★★★ 一般（只有架构概述）	★★★★★ 优秀（含模块、里程碑、部署）	★★★★★ 良好（四种架构对比完整）	★★★★★ 优秀（含数据模型、分工、演进路线）
对约束的敏感度	★ 极低（完全忽略团队规模、时间、技术栈）	★★★★★ 极高（严格基于约束）	★★★★★ 高（基于校园场景）	★★★ 中等（考虑了团队规模，但忽略了技术栈约束）
可直接采用程度	★ 不可采用	★★★★★ 可直接参考	★★★ 需二次加工	★★★ 需大幅裁剪

四、团队辩论与决策

4.1 辩论要点

- （成员A）：**AI 直接推荐微服务让我意识到，如果不给约束，AI 会默认按"理想团队"假设输出。这验证了 Prompt 工程的重要性。
- （成员B）：**给出详细约束后，AI 输出质量最高，甚至主动帮我们排除了微服务和 IM 系统。这说明**约束条件越具体，AI 越能理解真实场景**。
- （成员C）：**对比多种方案的输出帮我理解了不同架构的优劣，但 AI 把 MVC 和分层架构并列对比有点概念混淆。需要人工甄别技术概念的准确性。
- （成员D）：**扮演架构师后 AI 输出最完整，但存在"炫技"倾向——推荐了太多企业级组件（RabbitMQ、XXL-JOB、MinIO），对 10 周学生项目明显过度。这说明**AI 倾向于展示全面能力，而非针对约束做减法**。

4.2 关键发现

- Prompt 策略显著影响输出质量：**直接提问策略输出质量最低，提供约束后质量最高
- AI 存在"默认偏见"：**默认推荐流行技术（微服务、Go、React），而非最适合的技术
- AI 不擅长做减法：**即使给定约束，AI 仍倾向于推荐更多组件（如成员D的 RabbitMQ、XXL-JOB），而非精简方案
- 人工审查不可替代：**需要人工判断技术概念准确性（如 MVC vs 分层架构）、组件必要性（如是否需要消息队列）

4.3 团队投票

成员	倾向方案	理由
A	分层单体架构（成员B方案）	最符合团队实际约束
D	分层单体架构（成员B方案）	成员D方案需大幅裁剪，成员B更务实
B	分层单体架构（成员B方案）	自己提问得到的回答，最了解细节
C	分层单体架构（成员B方案）	成员C的对比分析也支持单体架构

投票结果：全票通过采用分层单体架构

4.4 最终决策

选定架构：前后端分离 + 分层单体架构

核心决策依据：

- 1. 时间约束优先：10 周周期内必须完成可演示系统，微服务的基础设施搭建将严重挤压业务开发时间
- 2. 团队能力匹配：团队仅熟悉 Java 语言，需依靠 AI 大幅辅助学习，分层架构的学习曲线最平缓
- 3. 需求规模匹配：单校 ≤3000 并发用户，单体架构完全可满足性能需求
- 4. 风险可控：单体架构调试、测试、部署简单，适合学生项目的运维能力

为什么不选其他方案：

方案	不选理由
微服务架构（成员A推荐）	基础设施搭建需 2-3 周，4 人团队 10 周无法完成；需学习 Nacos、分布式事务等，学习成本极高
事件驱动架构（成员C分析）	异步链路不直观，调试困难；需消息队列中间件，10 周易出数据一致性问题
成员D的"架构师方案"	组件过多（RabbitMQ、XXL-JOB、MinIO、Elasticsearch），10 周难以落地；前端技术栈与团队 Vue 背景不匹配

4.5 待确认证题

基于辩论，以下技术细节需在后续 ADR 中明确：

- 1. 缓存策略：Redis 在 MVP 阶段是否必须引入？（成员B建议引入，但团队需评估学习成本）
- 2. 消息队列：成员D推荐了 RabbitMQ，但成员B明确放弃 IM 系统。是否需要异步消息处理（如订单超时取消）？
- 3. 前端技术栈：团队熟悉 Vue 3，但成员D推荐 React。是否坚持 Vue 3 + Vite？
- 4. 文件存储：本地存储 vs 阿里云 OSS？（涉及成本和学习成本权衡）

五、Prompt 策略效果总结

策略	适用场景	优点	缺点	对本次项目的价值
直接提问	快速获取灵感	简单快捷	输出与实际情况偏差大	✖ 低：输出完全不可用
提供详细约束	需要可落地方案	输出精准、务实	需要前期整理约束条件	✔ 高：直接作为项目参考
对比多种方案	需要理解技术选型原理	结构化对比、原理清晰	偏理论，缺乏具体技术栈	★ 中：帮助团队理解架构优劣
扮演角色	需要完整方案框架	输出最全面、专业	易过度设计，需大幅裁剪	★ 中：提供完整框架参考，但需人工减法

最佳实践结论：对于学生团队项目，"提供详细约束"策略输出质量最高、最可落地；"扮演角色"策略可作为补充获取完整框架，但必须人工做减法。

文档编制： LXY

审核状态： 讨论结果已定，作为 Phase 2 架构决策依据